

CSC404

Database Management Systems

Complete Lecture Notes

Lecture 1: Intro to Databases & DBMS · Lecture 2: Relational Model · Lecture 3: XML Data · Lecture 4: JSON Data · Lecture 5: Relational Algebra · Lecture 6: SQL ·
Lecture 7: Relational Design Theory · Lecture 8: Indexes · Lecture 9: Constraints & Triggers · Lecture 10: Transactions · Lecture 11: Authorization

By Stephen Ebuka Iheagwara | Based on Widom (2012)

LECTURE 1

Introduction to Databases

What DBMSs do and why they matter

What is a DBMS?

A Database Management System (DBMS) provides:

- efficient, reliable, convenient, and safe multi-user storage of and access to
- massive amounts of persistent data

Database systems are everywhere — they sit behind:

- Websites and web applications
- Banking and financial systems
- Telecommunications networks
- Scientific experiments and sensor networks

Seven Key Properties of DBMS

1. **MASSIVE** — handles terabytes of data, far beyond what fits in memory
2. **PERSISTENT** — data outlives the programs that create and process it
3. **SAFE** — data stays consistent despite hardware/software failures or malicious users
4. **MULTI-USER** — concurrent access via concurrency control (many users, same DB)
5. **CONVENIENT** — physical independence + high-level declarative query languages
6. **EFFICIENT** — thousands of complex queries/updates per second over terabytes
7. **RELIABLE** — 99.9999% uptime guarantee for mission-critical applications

Key Concepts in DBMS

DATA MODEL — how data is structured:

- Relational model (tables of rows and columns) — dominant in industry
- XML (hierarchical/tree structure)
- JSON (hierarchical)
- Graph (nodes and edges)

SCHEMA vs DATA — like types vs variables in programming

- Schema: structure (tables, columns, types) — set up in advance
 - Data: actual values stored — changes constantly
-
- ❖ DDL (Data Definition Language) — defines the schema structure
 - ❖ DML (Data Manipulation Language) — queries and modifies data (SQL)

Key People in a DBMS Environment

- **DBMS IMPLEMENTER** — builds the DBMS software itself (not our focus)
- **DATABASE DESIGNER** — designs tables, schemas, relationships for a specific application
 - A surprisingly difficult task for complex, real-world applications
- **DATABASE APPLICATION DEVELOPER** — writes programs that interface between
 - users and the database (CRUD operations, business logic)
- **DATABASE ADMINISTRATOR (DBA)** — manages, maintains, tunes, secures, backs up
 - the database environment; performance, security, reliability
 - Key tuning parameters: memory/buffer, connection/concurrency, query planner, indexes
 - Highly valued, important, and well-compensated role

LECTURE 2

The Relational Model

The 50-year foundation of all major database systems

Why the Relational Model?

- Proposed by E.F. Codd at IBM in 1970 — now over 50 years old
- Underlies ALL major commercial database systems (Oracle, PostgreSQL, MySQL, SQLite...)

Key advantages:

- Very SIMPLE model — easy to reason about
 - Queried with HIGH-LEVEL LANGUAGES — compact, expressive, declarative
 - EXTREMELY EFFICIENT implementations available
 - Foundation of a multi-billion-dollar global industry
- ❖ The relational model is NOT going away — it remains the standard

Basic Constructs of the Relational Model

- **DATABASE** = set of named relations (tables)
- **RELATION (TABLE)** — has a name; holds data as rows
- **ATTRIBUTE (COLUMN)** — named; has a type/domain (int, string, float...)
- **TUPLE (ROW)** — one record; has a value for each attribute
- **SCHEMA** — structural description: relation name + attribute types (set in advance)
- **INSTANCE** — actual content at a point in time (changes constantly)
- **NULL** — special value for 'unknown' or 'undefined'
- **KEY** — attribute (or set) whose value uniquely identifies each tuple
 - Used to identify specific tuples; enables efficient indexing
 - Used to reference tuples from other relations (no pointers in relational model!)

Querying Relational Databases

Steps in using a relational database:

1. Design schema → create with DDL
2. Bulk-load initial data from files, spreadsheets, etc.
3. Execute queries and modifications repeatedly

Relational databases support AD HOC queries in high-level language:

- Write compact queries without implementing retrieval algorithms
- Declarative: say WHAT you want, not HOW to get it

Two query languages introduced:

- RELATIONAL ALGEBRA — formal, theoretical foundation; uses Greek symbols (σ , π , $\times$, $\bowtie$)
- SQL (Structured Query Language) — actual implemented language; built on rel. algebra

❖ Queries return relations (closure) and can be composed (compositionality)

LECTURE 3

XML Data

Semi-structured hierarchical data representation

What is XML?

- Extensible Markup Language — a standard for data representation and exchange
- Originally designed for exchanging information on the internet
- Like HTML, but tags describe CONTENT (not formatting)
- **Three basic constructs:**
 - TAGGED ELEMENTS — opening/closing tags; can nest to form a hierarchy
 - ATTRIBUTES — metadata within an element tag (name='value' pairs; must be unique)
 - TEXT — string data; forms the leaf nodes of the XML tree
- **Streaming standards: SAX (push parser) and StAX (pull parser)**
 - Process huge XML files without loading the entire document into memory
 - SAX: register callbacks; very memory-efficient
 - StAX: you control the loop; supports both reading and writing

Relational Model vs XML

Relational Model

- Tables (rows and columns)
- Schema required and fixed in advance
- Unordered (use ORDER BY for ordering)
- Simple, expressive query languages (SQL)
- Mature, native implementations
- Atomic types in attributes
- Strict uniformity across rows

XML

- Hierarchical / tree / graph structure
- Self-describing — schema optional/flexible
- Implied order from document/stream
- Query still settling (XPath, XQuery, XSLT)
- Usually a layer over relational systems
- Can mix elements, attributes, text
- Irregular structure allowed (e.g. optional elements)

Well-Formed vs Valid XML

WELL-FORMED XML — adheres to three basic structural requirements:

1. Single root element
 2. Matched tags and proper nesting (no interleaving)
 3. Unique attribute names within each element
- Checked by an XML parser; parsed XML output used by DOM or SAX/StAX

VALID XML — additionally conforms to a content-specific schema:

- DTD (Document Type Descriptor):
 - Grammar-like language; specifies elements, attributes, nesting, ordering
 - Introduces ID and IDREF(s) — untyped pointers within a document
 - Can be internal or external to the XML file
- XSD (XML Schema Description):
 - More powerful than DTD; supports typed pointers
 - Can check that IDREF points to correct element type

DTD Constructs & ID/IDREF

DTD grammar-like notation:

- Book* — zero or more Book elements (Kleene *)
- Author+ — one or more Author elements (positive Kleene +)
- (Title, Authors, Remark?) — sequence with optional Remark (?)
- #PCDATA — parsed character data (leaf text string)
- CDATA — attribute string value
- #REQUIRED / #IMPLIED — mandatory / optional attribute

ID and IDREF attributes:

- ID — special attribute type; value must be globally unique in the document
- IDREF — refers to one ID value (acts like a pointer to that element)
- IDREFS — space-separated list of one or more ID values
- ⚠ DTD pointers are UNTYPED — cannot specify which element type to point to
- XSD supports TYPED pointers — more specific constraints

LECTURE 4

JSON Data

The modern standard for semi-structured data interchange

What is JSON?

- JavaScript Object Notation — most recent of the three data models
- Originally for serialising JavaScript objects; now language-independent
- Human-readable; widely used for data interchange between systems
- Parsers available in virtually every programming language

THREE BASIC ATOMIC TYPES:

- Numbers, Strings, Booleans (true/false), null

TWO COMPOSITE TYPES (defined recursively):

- OBJECT — set of label-value pairs, enclosed in { }
 - e.g. { "ISBN": "978-0", "Price": 85, "Edition": "3rd" }
- ARRAY — ordered list of values, enclosed in []
 - e.g. [{"First_Name": "Joy"}, {"First_Name": "Ann"}]
- Values inside objects/arrays can themselves be objects or arrays

XML vs JSON — Comparison

XML

- More verbose (closing tags, attributes)
- More complex specification
- DTD / XSD for schema validation
- Older, more mature tooling
- Streaming: SAX / StAX standards
- Impedance mismatch with programming languages
- XPath / XQuery / XSLT query languages
- W3C-standardised across languages

JSON

- More concise (no closing tags)
- Simpler specification — easy to learn
- JSON Schema (less widely used)
- Newer; fewer mature tools
- Streaming via library-specific parsers
- Direct mapping to objects/dicts/arrays in most langs
- No widely-used standard query language yet
- De-facto standard for REST APIs and NoSQL

JSON Schema & Validation

- JSON Schema — expressed in JSON itself (schema file is a .json file)
- Constrains structure beyond basic syntactic correctness:
- Type constraints: 'type': 'integer', 'string', 'array', 'object', 'boolean'
- Range constraints: 'minimum': 0, 'maximum': 200
- Pattern constraints (strings): 'pattern': '^ISBN.*'
- Array length: 'minItems': 1, 'maxItems': 10
- Required properties: 'required': ['First_Name', 'Last_Name']
- Enumeration: 'enum': ['January', 'February', 'March']
- Multiple allowed types: 'type': ['integer', 'string']
- Strict mode: 'additionalProperties': false
- Labels in JSON objects should be unique (most parsers enforce this)
- Use arrays, not duplicate keys, for repeated values

LECTURE 5

Relational Algebra

The formal foundation of SQL — Select, Project, Join, Set Operators

What is Relational Algebra?

- Formal query language — the theoretical underpinning of SQL
- Operates on SETS of tuples and returns RELATIONS as results (closure)
- Example database used throughout:

College(cName, state, enrollment)

Student(sID, sName, GPA, sizeHS)

Apply(sID, cName, major, decision)

Core operators:

- σ Select — picks rows satisfying a condition
- π Project — picks specific columns
- $\times$ Cross-product — every combination of tuples from two relations
- $\cup$ Union · $-$ Difference · $\cap$ Intersection (set operators)
- ρ Rename — renames schema of a relation
- $\bowtie$ Natural Join · $\bowtie_{\theta}$ Theta Join (abbreviations, no new power)

Select, Project & Cross-Product

- SELECT σ — filters rows: $\sigma_{GPA>3.7}(\text{Student})$
 - Subscript = condition; applied to rows only
 - Multiple conditions with $\wedge$ (AND) / $\vee$ (OR)
- PROJECT π — selects columns: $\pi_{sID,decision}(\text{Apply})$
 - Eliminates duplicates (set semantics — unlike SQL!)
- CROSS-PRODUCT $\times$ — combines every pair of tuples:
 - $\text{Student} \times \text{Apply} \rightarrow S \times A$ tuples, with all columns from both
 - Disambiguate same-name columns: Student.sID , Apply.sID

Composing them:

- $\pi_{\{sName,GPA\}}\{\sigma_{\{sizeHS>1000 \wedge major='CS' \wedge decision='N'\}}(\text{Student} \times \text{Apply})\}$
- $\rightarrow$ Names and GPAs of CS-rejected students from large high schools

Natural Join, Theta Join & Rename

NATURAL JOIN $\bowtie$ = cross-product + equality on same-name attributes + remove duplicates

- Automatically matches sID from Student and Apply
- Simpler notation than explicit cross-product + select
- Does NOT add expressive power (can always rewrite with σ , π , $\times$)

THETA JOIN $\bowtie_{\theta}$ = cross-product filtered by a condition θ

- $E1 \bowtie_{\theta} E2 \equiv \sigma_{\theta}(E1 \times E2)$
- What database systems actually implement — 'join' usually means theta join

RENAME ρ — renames relation and/or attributes

- Essential for self-joins (same relation used twice)
- Essential to unify schemas before union/intersection
- Adds genuine expressive power — some queries are impossible without it
- Example: pairs of colleges in the same state requires renaming College twice

Set Operators: Union, Difference, Intersection

UNION $\cup$ — combines tuples from two relations (schemas must match!)

- Use Rename ρ first to unify schemas
- Duplicates eliminated (set semantics)
- Example: list of all college names + all student names

DIFFERENCE $-$: tuples in E1 but not E2 (very useful!)

- Example: IDs of students who did NOT apply anywhere
- $\pi_{\{SID\}}(Student) - \pi_{\{SID\}}(Apply)$

INTERSECTION $\cap$: tuples in both E1 and E2

- Does NOT add expressive power:
- $E1 \cap E2 = E1 - (E1 - E2)$ (rewrite using difference)
- $E1 \cap E2 = E1 \bowtie E2$ (when schemas are the same)

Expression trees and **Assignment Statements** are alternative notations

LECTURE 6

SQL

The language of relational databases — from basics to advanced features

SQL Basics — SELECT Statement

Basic form: SELECT $A_1, A_2, \dots$ FROM $R_1, R_2, \dots$ WHERE condition

- SELECT $\rightarrow$ like the project operator π
 - FROM $\rightarrow$ relations (cross-product if multiple, separated by commas)
 - WHERE $\rightarrow$ filter condition (like the select operator σ)
-
- SQL uses MULTI-SET semantics (bags) — duplicates are allowed by default
 - Relational algebra uses SET semantics — no duplicates
 - Use SELECT DISTINCT to eliminate duplicates

Useful clauses:

- ORDER BY *col* [ASC|DESC] — sort results
- LIKE '%*bio*%' — pattern matching on strings. This illustration matches strings that contain “*bio*”
- SELECT * — return all attributes
- *col* * 1000 AS *newName* — arithmetic and **column aliases**

Table Variables & Set Operators in SQL

- TABLE VARIABLES (aliases in FROM clause):
 - Make queries readable; enable self-joins (same table, two roles)
 - FROM Student S1, Student S2 WHERE S1.GPA = S2.GPA AND S1.sID < S2.sID
- SET OPERATORS (work on relations with compatible schemas):
 - UNION — eliminates duplicates; UNION ALL — keeps duplicates
 - INTERSECT — tuples in both results
 - EXCEPT — tuples in first result but not second
- Subqueries in WHERE clause:
 - IN / NOT IN — test membership in subquery result
 - EXISTS / NOT EXISTS — test whether subquery is empty
 - ALL — condition holds against every element of subquery
 - ANY — condition holds against at least one element
 - Correlated subqueries — inner query references outer query variable

SQL JOINS

INNER JOIN (default): $R \Join S \text{ ON condition} \equiv \text{theta join } \bowtie_{\theta}$

- `SELECT ... FROM Student JOIN Apply ON Student.sID = Apply.sID`

NATURAL JOIN: $R \Join S$

- Equates all same-named columns; removes duplicate columns

INNER JOIN USING: $R \Join S \text{ USING } (sID)$

- Explicit attributes to equate (like natural join but more controlled)

LEFT OUTER JOIN: $R \Join_{\text{left}} S \text{ ON condition}$

- All tuples from R; unmatched rows padded with NULL for S columns

RIGHT OUTER JOIN / FULL OUTER JOIN — similar, or both sides padded

- ❖ JOIN conditions can go in ON clause (join hint) or WHERE clause (filter hint)
- ❖ None of these add expressive power — useful notation and performance hints

Aggregation: GROUP BY & HAVING

Aggregate functions: COUNT(*), COUNT(DISTINCT col), SUM, AVG, MIN, MAX

GROUP BY — partition relation into groups, compute aggregate per group:

- SELECT cName, COUNT(*) FROM Apply GROUP BY cName
- Non-aggregated SELECT attributes MUST appear in GROUP BY

HAVING — filter on aggregate values (applied after grouping):

- HAVING COUNT(*) < 5
- WHERE applies per-tuple BEFORE grouping; HAVING applies per-group AFTER

NULL values and aggregation:

- COUNT DISTINCT ignores NULLs; SELECT returns NULLs; three-valued logic
- WHERE expressions evaluate to TRUE, FALSE, or UNKNOWN with NULLs
- IS NULL / IS NOT NULL — check for null values explicitly

Data Modification Statements

INSERT — add new tuples:

- `INSERT INTO Table VALUES (v1, v2, ...)`
- `INSERT INTO Table SELECT ... FROM ... WHERE ...` (insert query results)

DELETE — remove tuples:

- `DELETE FROM Table WHERE condition`
- Condition can include subqueries and aggregation over other tables
-  Some databases don't allow subquery on same table being deleted from

UPDATE — modify existing tuples:

- `UPDATE Table SET col = expression WHERE condition`
- `UPDATE Table SET col1 = e1, col2 = e2, ... WHERE condition`
- RHS expression can itself be a subquery (single-value subqueries)

- ❖ Power of **INSERT + SELECT**: bulk operations on derived data
- ❖ Power of **DELETE + subquery**: conditional multi-table deletions

LECTURE 7

Relational Design Theory

Functional Dependencies · BCNF · Multivalued Dependencies · 4NF

Motivation: Design Anomalies

A 'mega' relation with all attributes in one table causes:

- REDUNDANCY — same fact stored multiple times
- UPDATE ANOMALY — update one fact, miss others → inconsistency
- DELETION ANOMALY — deleting one fact may delete unrelated data

Example: Apply(SSN, sName, cName, HSname, HScity, hobby)

- Student with 2 high schools, 2 hobbies, 3 colleges → 12 tuples
- Every fact repeated: name×12, HS×6, hobby×6, college×4

Solution: Design by Decomposition

- Specify formal properties (FDs, MVDs)
- System automatically decomposes into BCNF / 4NF relations
- I.e., if FD and MVD properties are specified,, the system automatically decomposes into BCNF and 4NF, respectively
- Guarantees: no anomalies + lossless join (no information lost)

Functional Dependencies & BCNF

FUNCTIONAL DEPENDENCY $A \rightarrow B$: same A values $\rightarrow$ same B values (for all tuples)

- Read as “A functionally determines B”
- Based on real-world knowledge; all instances must satisfy declared FDs
- Example: SSN $\rightarrow$ sName (each SSN belongs to one student name)

BCNF: every nontrivial FD must have a KEY on its left-hand side

- If $A \rightarrow B$ but A is not a key $\rightarrow$ two tuples with same A can exist $\rightarrow$ redundancy

BCNF Decomposition Algorithm:

1. Compute keys via closure ($\bar{A}^+$ = all attributes $\rightarrow \bar{A}$ is a key)
2. Find violating FD: $A \rightarrow B$ where A is not a key
3. Decompose into $R1=(A \cup B)$ and $R2=(R' - B) \cup A$
4. Repeat until all relations in BCNF

FD Rules: Splitting, Combining, Trivial-dependency, Transitive

Multivalued Dependencies & 4NF

MVD $A \twoheadrightarrow B$: given a value of A, B values are independent of all other attributes

- Read as “A multi-determines B”
- Causes exponential blowup: m colleges $\times$ n hobbies = m $\times$ n tuples (want m+n)
- BCNF doesn't address this — relation can be in BCNF but still have MVD problems

4NF: every nontrivial MVD must have a KEY on its left-hand side

- 4NF $\subset$ BCNF (every 4NF relation is in BCNF, not vice versa)

4NF Decomposition Algorithm (same structure as BCNF):

1. Compute keys using FDs
2. Find violating MVD: $A \twoheadrightarrow B$ where A is not a key
3. Decompose into $R_1=(A \cup B)$ and $R_2=(R' - B) \cup A$
4. Repeat until all relations in 4NF

Normal form hierarchy: 4NF $\subset$ BCNF $\subset$ 3NF $\subset$ 2NF $\subset$ 1NF

LECTURE 8

Indexes

Accelerating queries with data structures

What is an Index?

- An INDEX is a data structure built on top of a table's data to speed up lookups
 - Without index: full table scan — $O(n)$ time
 - With index: direct lookup — $O(\log n)$ or even $O(1)$ time
- The three most important things about a database: performance, performance, performance
- Underlying structures:
 - B-Tree / B+-Tree — balanced tree; supports range queries and equality lookups (E.g., $A \leq B$) or (E.g., $A = B$)
 - Hash index — equality lookups only (E.g., $A = B$); very fast ($O(1)$)
- Indexes are built on one or more columns and stored separately from table data
- The database query optimizer decides whether to use an index for each query
- Most systems create indexes automatically for PRIMARY KEY and UNIQUE constraints

Index Functionality & Utility

FUNCTIONALITY — what queries benefit from an index on column(s) A:

- Equality lookups: WHERE A = 5
- Range queries: WHERE A BETWEEN 10 AND 50
- Ordering: ORDER BY A (index already stores sorted order)
- Joins: if joining on A, index on A in either table speeds the join

UTILITY — when is an index actually helpful?

- When the query is selective (returns small fraction of table)
- When the table is large (full scan is expensive)
- When queries are frequent on that column

DOWNSIDES of indexes:

- Extra storage space (the index structure itself)
- Overhead on INSERT, UPDATE, DELETE (index must be maintained)
- Too many indexes → write operations slow down significantly

Picking Indexes & SQL Syntax

CHOOSING WHICH INDEXES TO CREATE:

- Build on columns frequently appearing in WHERE, JOIN, ORDER BY
- Prefer indexes on high-selectivity columns (many distinct values)
- Consider your query workload — profile before indexing
- Physical Design Advisor: feed queries to the optimizer; it suggests indexes
- Use EXPLAIN / EXPLAIN ANALYZE to see if optimizer uses an index

SQL SYNTAX:

- `CREATE INDEX idx_name ON Table(col);`
 - `CREATE INDEX idx_name ON Table(col1, col2);` -- composite index
 - `CREATE UNIQUE INDEX idx_name ON Table(col);` -- enforces uniqueness
 - `DROP INDEX idx_name;`
- ❖ Each DBMS has its own syntax variations and index types (BTREE, HASH, GiST, ...)

LECTURE 9

Constraints & Triggers

Enforcing data integrity and automating reactions to change

Integrity Constraints

INTEGRITY CONSTRAINTS — assertions that data in the database must always satisfy

- Checked on every INSERT, UPDATE, DELETE (not just at query time)
- Caught early → bugs found before they propagate through the application

Types of constraints:

- **NOT NULL** — attribute may not be null
- **PRIMARY KEY** — unique identifier; implies UNIQUE and NOT NULL
- **UNIQUE** — all values of a column (or column combination) are distinct
- **CHECK (condition)** — arbitrary condition on a tuple's own values
- **REFERENTIAL INTEGRITY / FOREIGN KEY** — value must exist in another table
- **GENERAL ASSERTION** — condition involving multiple tables (rarely supported)

Enforcement options: DEFERRED (at commit) vs IMMEDIATE (after each statement)

Referential Integrity (Foreign Keys)

FOREIGN KEY constraint: R.A must reference S.B

- Every non-null value in R.A must appear in S.B

Violation actions — what to do when referencing row would become dangling:

- ON DELETE RESTRICT — block the delete (default)
- ON DELETE CASCADE — delete referencing rows too
- ON DELETE SET NULL — set referencing column to NULL
- ON DELETE SET DEFAULT — set referencing column to default value
- ON UPDATE CASCADE / SET NULL / SET DEFAULT — same for updates

SQL syntax:

- FOREIGN KEY (sID) REFERENCES Student(sID) ON DELETE CASCADE

❖ **Referential integrity is the most commonly used constraint in practice**

TRIGGER is a procedure that executes automatically when a specified event occurs.

Components of a trigger:

- **EVENT** — INSERT, UPDATE, or DELETE on a specific table
- **CONDITION** — optional WHEN clause; trigger body runs only if condition is true
- **ACTION** — SQL statements or procedure calls to execute

Timing:

- **BEFORE trigger** — fires before the data change
- **AFTER trigger** — fires after the data change
- **INSTEAD OF trigger** — replaces the operation (useful for views)

Granularity:

- FOR EACH ROW — fires once per affected row
- FOR EACH STATEMENT — fires once per SQL statement

Use cases: simulate cascade, enforce complex constraints, audit logging

-  Watch for self-triggering, cycles, and conflicts between triggers

LECTURE 10

Transactions

ACID properties and isolation levels

Why Transactions?

Two independent requirements motivate transactions:

1. CONCURRENT DATABASE ACCESS:

- Multiple users/applications operating on the same data simultaneously
- Risk: interleaved operations cause inconsistency
- Examples: attribute-level, tuple-level, table-level, multi-statement inconsistency

2. RESILIENCE TO SYSTEM FAILURES:

- Hardware failures, power outages, software crashes
- Risk: partial updates leave database in inconsistent state

TRANSACTION is a sequence of one or more SQL operations treated as a unit.

- Either ALL operations commit (permanently saved) or ALL abort (rolled back)
- Concurrency goal: execute as if serial (one at a time) for correctness
- Failure goal: each transaction is atomic — all or nothing

ACID Properties

A — ATOMICITY: all operations in a transaction succeed or all are rolled back

- Transaction rollback (abort) undoes all partial changes

C — CONSISTENCY: database moves from one valid state to another

- All constraints and integrity rules must be satisfied after commit
- Application code is also responsible for maintaining logical consistency

I — ISOLATION: concurrent transactions appear to execute serially

- Each transaction sees a consistent snapshot; no interference from others
- Implemented via locking, MVCC (multi-version concurrency control)

D — DURABILITY: committed changes survive any subsequent failures

- Implemented via write-ahead logging (WAL) and recovery mechanisms

❖ Most systems default to READ COMMITTED isolation — trade-off between correctness and performance

Isolation Levels

READ UNCOMMITTED — can read uncommitted changes ('dirty reads'); no locks

- Fastest; least safe; rarely appropriate

READ COMMITTED (default in most DBs) — only read committed data

- Prevents dirty reads; allows non-repeatable reads

REPEATABLE READ — same query returns same result within a transaction

- Prevents **dirty** and non-repeatable reads; may allow phantom reads

SERIALIZABLE — full isolation; transactions appear to run one at a time

- Safest; most expensive; often the default in academic settings

READ ONLY transaction — hints to optimizer; no writes possible

- Allows less locking overhead; can be any isolation level

Isolation Levels

SQL:

- SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
- BEGIN; ... SQL statements ... COMMIT; / ROLLBACK;

EXAMPLE:

Insert Into Student [100 new tuples];

Commit;

T1

Concurrent with ...

Set Transaction Isolation Level Repeatable Read;

Select Avg(GPA) From Student;

Select Max(GPA) From Student;

Commit;

T2

LECTURE 11

Authorization

Controlling who can see and change what

Database Authorization

- Authorization controls who can access and modify database objects
- Privileges are granted or revoked by the database owner or administrators

TYPES OF PRIVILEGES:

- SELECT — read rows from a relation
- INSERT — add new rows
- UPDATE — modify existing rows (can be column-specific)
- DELETE — remove rows
- REFERENCES — create foreign keys referencing this table
- ALL PRIVILEGES — all of the above

SQL SYNTAX:

```
GRANT SELECT ON Student TO user1;
```

```
GRANT SELECT, INSERT ON Apply TO PUBLIC; -- ALL users
```

```
REVOKE UPDATE ON College FROM user2;
```

```
GRANT SELECT ON Student TO user3 WITH GRANT OPTION; -- can re-grant
```

Revoking & Managing Privileges

- **REVOKE** removes a previously granted privilege:
 - REVOKE SELECT ON `Student` FROM `user1`;
- **WITH GRANT OPTION** — allows recipient to grant the same privilege to others
- **CASCADE** — when revoking, also revoke any privileges granted by the revokee
- **RESTRICT** — revoke fails if the revokee has granted the privilege further
- **Privileges reside in a privilege graph:**
 - Each user holds privileges; some are marked WITH GRANT OPTION
 - Revoking with CASCADE propagates through the privilege chain

VIEWS as a security mechanism:

Grant SELECT on a VIEW (not the base table) to limit visible columns/rows

```
CREATE VIEW PublicStudents AS SELECT sName, GPA FROM Student;
```

```
GRANT SELECT ON PublicStudents TO user5;
```

- Role-based access control (RBAC) — group privileges into named roles

Course Summary

Lecture 1	DBMS Intro	Massive, Persistent, Safe, Multi-user, Convenient, Efficient, Reliable
Lecture 2	Relational Model	Tables, Attributes, Tuples, Keys, Schema vs Instance, Query Languages
Lecture 3	XML Data	Tags, Elements, Attributes, Well-Formed, DTD, ID/IDREF, XSD
Lecture 4	JSON Data	Objects, Arrays, Schema Validation, Flexibility, NoSQL usage
Lecture 5	Relational Algebra	σ Select, π Project, $\times$ Cross, $\bowtie$ Join, $\cup$ $\cap$ $-$, ρ Rename
Lecture 6	SQL	SELECT-FROM-WHERE, GROUP BY, Subqueries, JOINS, Aggregation, DML
Lecture 7	Design Theory	FDs, BCNF, MVDs, 4NF, Decomposition, Anomalies
Lecture 8	Indexes	B-Tree, Hash, CREATE INDEX, selectivity, query optimiser
Lecture 9	Constraints & Triggers	PK, FK, CHECK, UNIQUE, NOT NULL, BEFORE/AFTER triggers
Lecture 10	Transactions	ACID, Isolation Levels, Commit, Rollback, Concurrency
Lecture 11	Authorization	GRANT, REVOKE, WITH GRANT OPTION, Views, RBAC